

ZEQUI

AI Study & Growth Companion

Full Project Documentation • Features • Architecture • Engineering Research

Prepared from the supplied ZEQUI source archive

Core idea: turn AI from a single chat box into a multi-tool study companion — ask → understand → summarize → practice → track → focus → improve.

Confidentiality: API keys, tokens, private credentials, environment secrets and other sensitive implementation values are intentionally omitted.

1. Executive Overview

ZEQUI is a web-based AI study and growth companion built around a modular React + TypeScript interface. The code combines AI conversation with research, summarization, advice, companion mode, quiz generation, topic tracking, exam planning, focus tools, voice interaction and personalization.

The implementation shows a deliberate attempt to make learning continuous rather than isolated: a learner can ask a question, get an explanation, summarize material, generate a quiz, review mistakes, track recurring topics and plan around exams without leaving the same workspace.

Public-facing product promise: a focused AI workspace for studying, focus and personal growth.

2. Complete Feature Inventory

Feature	Implemented behavior	Purpose
AI Chat	Persona-aware conversation, message history, loading/error handling, voice input, text-to-speech, quiz generation and chat export.	Interactive learning and follow-up questions.
Research	Research query input; response parsed into a short answer, three key facts and suggested sources.	Fast research starting point.
Smart Summarize	Quick and Deep + Advice modes; paste notes or load a file; AI generates study-oriented output.	Condense long material.
Advice	Study-strategy and motivational guidance with saved messages and speech playback.	Practical study support.
Companion	Friendly conversational mode with quick replies, welcome interaction and speech.	Lower-friction AI companionship.
Quiz Mode	Generates five MCQs, records answers, scores results, explains mistakes and exports the quiz.	Active recall and self-testing.
Topic Tracker	Extracts keywords from questions and counts recurring topics locally.	Identify repeatedly visited topics.
Exam Countdown	Stores exams, calculates remaining time and surfaces focus suggestions using tracked topics.	Deadline-aware study planning.
Focus Timer	Dedicated focus panel starting at 25 minutes with timer controls.	Focused study sessions.
Study Stats	Stats panel available through the Tool Dock.	Progress-oriented workspace.
Profile	Persistent profile fields, bio, accent color, streak and level display.	Personalization.
Personas	Selectable AI behavior/personality profiles.	Different interaction styles for different study situations.
Themes	Dark/light theme and persistent accent color.	Personalized UI.
Voice	Browser SpeechRecognition input and speechSynthesis playback.	Hands-free/accessibility support.
Export	Chat and quiz exports through browser downloads.	Keep/share study material.
Persistence	localStorage-backed preferences, profile, exams, quiz history, weak topics and cache.	Continuity between sessions.

3. Architecture

The source is organized as a Vite-powered React + TypeScript frontend with feature components, reusable hooks/utilities and a Netlify serverless AI router.

Layer	Code evidence	Role
UI	React components + Tailwind CSS + Lucide React	Responsive interface and interaction states.
Features	ChatTab, ResearchTab, AdviceTab, CompanionTab, SmartSummarizeCard, QuizMode, TopicTracker, ExamCountdown, ToolDock, etc.	Separates major product workflows.
State	React state + custom hooks + localStorage	Manages UI state and lightweight persistence.
Utilities	file processing, sanitization, personas, cache and API configuration	Reusable cross-cutting behavior.
Serverless boundary	Netlify function /.netlify/functions/ai-router	Central AI request boundary.
AI providers	Gemini → Groq → Hugging Face fallback in router	Provider resilience.

High-level request path

User → React feature → request preparation → Netlify AI router
→ Gemini / Groq / Hugging Face → response → UI → optional study action

4. AI Routing & Resilience Research

The most notable backend engineering pattern is provider abstraction. The Netlify function defines separate calls for Gemini, Groq and Hugging Face and attempts them sequentially.

Each provider call is wrapped in an approximately eight-second timeout. The router continues after failures such as rate limits, server errors or timeouts, then returns the first successful response.

This means the frontend can use one AI endpoint while the server decides which configured provider is available. It also creates a central place for future provider changes, request validation and operational controls.

```
providers = [gemini, groq, huggingface]
for provider of providers:
  try:
    text = await withTimeout(provider.call(), 8000)
    return text
  except:
    continue
```

5. Research & Learning Design Reflected in the Code

The source does not contain a formal academic research report or controlled experiment. This section therefore documents the learning/product concepts that are **implemented in the software**, without claiming that the implementation itself proves educational effectiveness.

- **Active recall:** Quiz Mode turns source content into questions and requires the learner to retrieve answers.
- **Chunking:** Research output is separated into an answer, key facts and suggested sources.
- **Feedback:** Quiz scoring and mistake review show the learner what was missed.
- **Repetition signals:** Topic Tracker counts recurring keywords to identify subjects that keep returning.
- **Deadline awareness:** Exam Countdown connects learning activity with upcoming dates.
- **Focus support:** The Tool Dock exposes a dedicated timer without leaving the main workspace.
- **Personalization:** Personas, profile settings, theme and accent preferences adapt the interface/interaction.
- **Multimodal access:** Speech input and text-to-speech provide alternatives to typing/reading.

6. Detailed Workflow

- **Chat:** user enters a message → selected persona system prompt is added → AI request is sent → response is rendered → user can listen, continue, export or generate a quiz.
- **Research:** question → structured research prompt → AI response → parser extracts answer/facts/sources → UI displays the three sections.
- **Summarize:** notes/file → selected Quick or Deep mode → serverless AI router → generated result → scrollable summary panel.
- **Quiz:** learning content → AI instructed to return five JSON MCQs → quiz parsed → learner answers → score calculated → history stored → mistakes can be reviewed.
- **Exam planning:** exam name/date stored locally → countdown recalculated → upcoming exams shown → tracked weak topics can influence a focus suggestion.
- **Personalization:** profile/persona/theme/accent changes → local browser storage → settings remain available in later sessions.

7. File Processing & Security

The project contains a dedicated file processor with a maximum file-size rule, allowed-type checks, text extraction and PDF text handling. The summarization UI exposes file input for study material.

The codebase also includes sanitization helpers, including a DOMPurify-based utility. This is important because AI output and user-provided content should not be treated as trusted HTML.

The serverless router reads provider credentials from environment variables rather than requiring the values to be documented in the portfolio.

Security area	Current code evidence
Secrets	Serverless router reads provider keys from process.env.
Request validation	Router checks that messages exists and is an array.
Timeout control	Provider calls use an eight-second timeout wrapper.
Output safety	Sanitization helpers are present for displayed content.
Local data	Profile, exams, quiz history, preferences and cache use browser storage.

8. UX & Interface Research

- Five main tabs: Chat, Summarize, Research, Advice and Companion.
- Desktop layout includes a study sidebar with Topic Tracker and Exam Countdown.
- Mobile Bottom Navigation provides a mobile-oriented navigation path.
- Right-side Tool Dock opens Focus and Stats overlays.
- Top bar combines ZEQUI branding, theme control, persona selection and profile access.
- Toast notifications provide success/error/info feedback.
- Glass-panel styling, cyan/violet accents and animated branding create a consistent visual identity.
- Responsive design is built around Tailwind utility classes and breakpoint-based layouts.

9. Technology Stack

Technology	Role
React 18	Component-based UI and state management.

Technology	Role
TypeScript	Typed components, hooks, interfaces and feature data.
Vite	Frontend development and build tooling.
Tailwind CSS	Responsive styling, layout and themes.
Lucide React	Icons and interface controls.
Netlify Functions	Serverless AI routing.
DOMPurify	Sanitization helper.
jsPDF / pdfjs-dist	PDF-related project capabilities/dependencies.
Browser Web APIs	localStorage, FileReader, SpeechRecognition, speechSynthesis and downloads.

10. Engineering Findings From the Current Source

- **Modular architecture:** major capabilities are separated into dedicated components and hooks, which makes the product easier to extend.
- **Central AI boundary:** the Netlify router provides a single backend entry point for multi-provider AI fallback.
- **Two AI access patterns exist:** the repository contains direct Groq helper code as well as the newer serverless router. Consolidating production calls around the router would simplify the architecture and secret handling.
- **File validation should be unified:** the summarizer UI advertises more extensions than the central file processor currently lists. These rules should be made consistent.
- **Research sources are suggested, not verified:** the Research tab creates Google search links from source names; it does not itself retrieve or validate citations.
- **Usage/Pro logic is partly product scaffolding:** local flags and a placeholder payment link are present, so real billing/authentication would need a backend/account system.
- **Persistence is browser-local:** the product can remember useful study state on the same browser, but this is not cloud synchronization.
- **Strong extension points:** personas, tabs, hooks, utilities and the Tool Dock provide natural places for future capabilities.

11. Recommended Next Engineering Steps

- Route all production AI calls through the serverless router and isolate/remove legacy direct-client provider calls.
- Synchronize file input acceptance with actual validation/processing support.
- Add real web retrieval and citation verification if Research is intended to provide source-backed answers.
- Replace local-only Pro/usage flags with authenticated server-side entitlements for production monetization.
- Add automated tests for router fallback, timeouts, file validation, quiz JSON parsing, cache expiry and topic extraction.
- Add server-side rate limiting/authentication for a public AI endpoint.
- Add clear controls for deleting locally stored profile, cache, quiz and exam data.
- Add cloud sync only if cross-device continuity becomes a product requirement.

12. Portfolio / LinkedIn Summary

ZEQUI — AI Study & Growth Companion is a React + TypeScript learning workspace that combines AI chat, research, summarization, advice, companion mode, quiz generation, topic tracking, exam countdowns, focus tools,

voice interaction and personalization.

The project explores modular frontend architecture, browser persistence, AI provider abstraction, serverless routing, fallback handling, file processing, sanitization and study-oriented UX.

The main product idea: instead of treating AI as a single chat interface, ZEQUI creates an integrated learning loop: **ask** → **understand** → **summarize** → **practice** → **track** → **focus** → **improve**.

Appendix — Source Scope & Accuracy

This document was produced by inspecting the supplied ZEQUI project archive, including the main application, feature components, hooks, utilities, configuration, package manifest and Netlify AI router.

It describes the capabilities and architecture visible in the source at the time of inspection. It does not claim that every feature is production-tested, that AI answers are automatically fact-checked, or that every external service is currently configured and operational.

Confidential implementation values such as actual API keys, tokens, private credentials and environment secrets are intentionally not reproduced.